

Types for a Lisp on the BEAM

A sound-warnings-only type system with a logic tier — the design, the tradeoffs, and the running demos that prove each piece

applied design · every claim below is backed by a running demo

2026-08-28

What this is, from zero

beam-lisp is a Lisp that compiles to the BEAM — the virtual machine under Erlang and Elixir. It has persistent data structures, macros, multimethods, protocols, atoms, and processes; it interoperates with Elixir modules directly; its standard library (`core`) is written in itself. It is a *dynamic* language: values carry their types at runtime, and nothing in the compiler asks about them.

This document is the applied design of a **type system for beam-lisp**: what it is, what it deliberately is not, how each piece works, and — the point of the exercise — the running demonstration of each piece. Nothing here is aspirational. Every section names the demo that proves it, under `examples/typing/`, each of which prints `PASS` today.

THE ONE-SENTENCE VERSION

A checker that reads beam-lisp source and emits **only warnings it can prove**, at the positions the user wrote, with unknowns deferred rather than guessed — plus a logic tier (an SMT solver, a logic-programming engine, a datalog database of the code itself) that answers the questions a tag lattice cannot.

Reading guide

No background in type systems is assumed. §2 builds the vocabulary. §3 covers what Elixir — beam-lisp’s host ecosystem — does about types and what of it applies. §4 states the contract everything else obeys. §5–§10 are the design proper, layer by layer, each with its evidence. §11 is the honest list of what this design gives up. §12 is where it goes next. §13 is references, including why the SMT backend is z3 and not something smaller.

Type systems: the landscape in five pages’ worth of one page

A **type** is a claim about what shape a value has: “this is an integer”, “this is a map with a `:name` key”. A **type system** is machinery that checks whether those claims can hold — *before* the program runs.

Static systems (Haskell, Rust, Lean) prove claims at compile time and **reject** programs they cannot prove. Their guarantee is soundness: if it compiles, the claims hold. The price is that provability is the gate — a correct program the checker cannot follow does not compile.

Dynamic systems (Python, Ruby, Clojure, beam-lisp today) check claims at runtime, per operation. Nothing is rejected up front; a wrong claim becomes a crash at the moment of use — possibly in production, possibly never (dead code is never checked at all).

Gradual systems (TypeScript, Elixir’s emerging system, this design) sit between: they check statically what they can, stay silent where they cannot, and never reject. The design axis that matters is not static-versus-dynamic; it is what the checker does when it does not know.

THE AXIS THIS DESIGN IS BUILT ON

When the checker cannot prove something, it has three options: **reject** (static soundness), **guess** (unsound warnings — false positives), or **defer** (record the question, answer it when more information arrives, stay silent if it never does). beam-lisp’s checker only ever defers.

Two more terms used throughout. **Inference** means the checker derives types from the code itself; **annotation** means the programmer declares them (`( ^{:args [int] :ret int} )`). This design does both: annotations where the programmer knows, inference where the code proves, silence everywhere else. And **delaboration**: whenever the checker shows the user a type, it shows it in the user’s notation — never in the machine’s internal representation.

What Elixir does, and what of it applies

Elixir is gradually acquiring a static type system (the set-theoretic types work by Castagna, Duboc, and de Azevedo, landing release by release since 1.17). Its types are **set-theoretic**: union, intersection, and negation are first-class, so `integer()` and `not 0` is a type. Its guarantee is deliberately **not** full soundness: Elixir’s system aims to catch definite bugs in gradually typed code without breaking the ecosystem’s dynamic idioms.

What beam-lisp inherits from this:

- **The philosophy.** Sound-warnings-only, never reject, unknown stays silent. Elixir’s designers converged on the same contract for the same reason: a Lisp/BEAM codebase is full of code a checker cannot follow, and a checker that cries wolf gets turned off.
- **The culture of the BEAM.** Dialyzer (Erlang’s discrepancy analyzer) has always been a **warning** tool, not a gate. Twenty-five years of BEAM practice says warnings-not-gates is the right shape.

What beam-lisp cannot reuse:

- **Elixir’s types themselves.** They are defined over Elixir’s AST and Elixir’s forms — guards, pattern clauses, protocols all differ. The machinery does not transfer; only the philosophy does.
- **Set-theoretic negation.** A beautiful theory with real costs (decidability pressure, solver complexity). beam-lisp’s lattice is a plain set-of-tags lattice: union and intersection, no negation. §11 discusses what this gives up.

The type system is independent of the compilation target: types are computed from source, checked, and **erased** before codegen. No beam file changes; `infer_signatures: false` stays; a program the checker hates runs exactly as before.

Because — beam-lisp is a Lisp: its code is written by macros and, increasingly, by language models. Both produce code that is correct more often than it is **followable**. A gate would reject good code; guesses would drown real warnings in false ones.

The contract

Everything in this design obeys four rules. They are stated here once and referenced everywhere.

THE CONTRACT

1. **Sound warnings only.** Every warning is a proof: the checker shows the two types and why they cannot meet. If it cannot prove a fault, it does not speak.
2. **Unknown $\rightarrow$ deferred, never guessed.** A call to a function the checker cannot see is recorded as a **deferred constraint** and retried when a later namespace defines the callee; at the end of the require-DAG, what is still unknown is listed as silent-unknown, never warned. (§8.5, and the first real client in §5.6.)
3. **any stays silent.** The top of the lattice is not an error. Unannotated, unseeded, unprovable code checks clean. This is a feature demonstrated repeatedly in the demos: the checker’s silence is where the dynamic language lives.
4. **Positions are the user’s.** Every warning carries the line and column the user wrote — including through macros, whose expansions are never blamed for what the user typed. (§5.2.)

The contract is not aspirational: P10 checked the whole datum closure — 31 files, 5,789 evidence entries — and reported **zero** warnings after three checker bugs it exposed were fixed at their roots (research/p10_scale). The false-positive rate on real code is the design’s most guarded number.

The applied design, layer by layer

The tag lattice and the annotation pipe

Tags are beam-lisp’s runtime type names: `:int`, `:string`, `:keyword`, `:map`, `:fn`, thirteen in all. A **type** in this system is a *set of tags* — “this value is an int or a string” is `#{:int :string}`. The lattice operations are set operations: union, intersection (meet), difference. The top of the lattice is all thirteen tags (`any`); the bottom is the empty set (`never` — a provably dead value).

Annotations ride metadata, the Lisp-native surface:

```
(defn ^{:args [int] :ret int} double-it [x] (* x 2))
```

 Clojure

The annotation is **data**, **never evaluated**, attached to the name, and it survives the whole pipeline: reader $\rightarrow$ compiler $\rightarrow$ environment metadata $\rightarrow$ AOT-compiled module. That pipe was the first thing proven (P0) — a type system whose annotations evaporate at load time is a decoration.

Demo 01 (`examples/typing/01_check_demo.bl`) annotates a module for real, runs it, then checks a never-run buggy snippet: three warnings, each at the user-written position, each elaborated — `double-it: argument 1 is (:string), declared (:int)`. Hovering the evidence table at that position answers `(:int)`.

Unknown functions the lattice cannot see are seeded from two curated tables: `core-seeds-v2` (41 core functions, runtime-verified — `keys` returns a lazy seq, `sort` a list) and `host-seeds` (22 Elixir interop entry points). The seed discipline is **sound-conservative**: a doubtful signature is left unseeded — which yields `any` — which the contract keeps silent. Seeds are where false positives would breed; they are deliberately incomplete.

Positions through macros: the InfoTree move

A checker that expands macros loses the user's positions; a checker that walks source literally misunderstands macro semantics. The answer, measured in P2:

A fixed whitelist of common macros (`when` , `cond` , `and` , `or` , `->` , `->>`) is **walked structurally, never expanded**. Threading macros are desugared the way Lean's InfoTree does it: each synthesized call node is stamped with the position of the step the user actually wrote. Unknown macros are expanded once with the real compiler's `macroexpand_1`, checked for identity against the **original** node, and attributed to the outer position. **Because** — expansion is where positions die; non-expansion is where semantics die. The whitelist keeps both alive for the forms that dominate real code, and the oracle keeps the door open for the rest.

The same walk builds the **evidence table**: a map from every `[line col]` to the proven type there. That table is what hover reads, what the promotion engine (§7) reads, and what an editor would read. It is an invariant of the design (invariant 9): the checker never computes a type it does not also record with its position.

Guards, clauses, and the claim/narrow asymmetry

Multi-clause functions with `:when` guards taught the checker its most quoted rule:

CLAIM ≠ NARROW

For **coverage** (is this clause reachable?), an `and`-guard claims nothing — both sides must hold, so it excludes nothing provable. For **narrowing** (what is `x` inside the body?), an `and`-guard narrows by the **meet** of its parts — both sides hold there. Two different functions, deliberately: `guard-claim` and `guard-narrow`. A checker that uses one for the other manufactures false positives.

P3 checked 204 lines of guard-heavy code and a destructuring corpus with **zero warnings, hand-audited** (audit table in `research/p3_guards/README.md`). A synthetic dead clause — a second `pos?` branch after an exhaustive first — warns `clause unreachable` at its exact position. The audit, not the count, is the evidence: zero warnings is only good news when a human agrees the code was clean.

Open worlds: multimethods, protocols, and the fingerprint

Multimethods and protocols are **open**: any namespace can add a method, any time. A check that depends on the method set is therefore valid only **at a fingerprint** — the set of `(multi, dispatch-value)` pairs visible when the check ran.

P4 demonstrated the full lifecycle: a consumer of a two-method `area` multimethod warns (sound — nothing returns a string yet); a string-returning `:default` method arrives; the fingerprint changes 2 → 3; the warning is **revoked**. P5 showed the same mechanism covers protocols, with one extra subtlety: protocol dispatch tags (`:integer` , `:binary` , struct modules) are not lattice tags, so a partial map bridges them and unmapable tags claim nothing.

Two consequences. Extension can only **revoke** warnings (a wider method set kills provability, never creates it), so the lattice’s soundness survives openness. And the fingerprint belongs in the AOT cache keys: a cached analysis is valid exactly when the fingerprint it was computed under still holds. One mechanism, both features.

Mutation, atoms, and servers

Atoms are typed by their initial value: `(def a (atom 0))` gives `a : atom(int)`; `swap!` / `reset!` must meet the inner type; `deref` yields it. One special case earns its keep:

A nil-initialized atom types as `atom(any)`. The checker gives up precision exactly where the idiom makes precision wrong.

Because — `(atom nil)` with a later `reset!` is the set-later idiom — typing it `atom(never)` would flag every legitimate write, a false-positive factory.

Reloads need no fingerprint machinery (contrast §5.4): type tables are **derived**, rebuilt per load, keyed exactly like the environment’s re-intern. `defserver` state is typed by its init, and every reply position must meet it — a `(noreply "not-a-map")` under map state warns at its line (P7, 4/4 cases).

Map shapes and the schema bridge

P8 graduated maps from a single `:map` tag to **shapes**: `[:shape {:name #{:string} :age #{:int}}]` sits beside plain tags in the lattice. Union is set union (so `{:ok int} | {:err string}` is one type); meet is per-key; keyword-call and `get` narrow to the key’s type. Two soundness rules were found by being wrong first:

- **Partial presence means nil.** If a key is absent from one union member, reading it can yield nil — so nil joins the read’s type. The first spike forgot this and was unsound; the fix is the rule.
- **“Key never present” warns only on literal-derived shapes.** A literal in source is exact by construction, so a key the literal never has is provably absent — zero false positives **by construction**, not by testing. Param-bound maps stay shapeless and silent.

P9 bridges the database: a datom schema installed from a literal transaction (the dominant style) is statically readable, so query result columns type from the schema’s `valueType` — string → `:string`, long → `:int`, ref → `:int`, and unknown attributes yield `any` plus a deferred constraint (the contract’s first real client). This is the multiplier: shapes no longer originate only at literals — they arrive through queries.

Scale

P10 is the row that turns “works on demos” into “works on the repo”:

```
THE DATOM CLOSURE — 31 FILES, EVERY EVIDENCE ENTRY
354 ms cold · 5 ms warm (hash-skip) · 5,789 evidence entries · 0 warnings
```

The first run reported 11 warnings; every one was a checker bug (a dispatch branch that let `ns` forms fall through to the walker, an under-wide `get` seed, higher-order seeds that refused keyword-callable arguments). All three were fixed at the root in `priv/typed.bl`, and the lesson is now a rule: **over-narrow seeds are false-positive factories** — when in doubt, unseeded.

The logic tier

The tag lattice answers **structural** questions (can these tags meet?). Three engines answer what it cannot, with a strict division of labour: no overlaps, no fallbacks.

NON-OVERLAP AS A DESIGN RULE

The tag lattice owns **structure**. z3 owns **decisions** (is this rule provable?). miniKanren owns **relations** (what term fits this hole?). datom owns **the codebase as data** (who calls this?). A question is asked of exactly one engine. The engines never compete, so their answers never need reconciling.

z3 as a native call — and why not a baby solver

z3 runs as an external process behind a port driver, wrapped as a native call — the same pattern as a NIF or **defnative**. The protocol lessons (one **(reset)** per query, line-scanned answers, paren-balanced model reads) are encoded once in **lib/beam_lisp/z3_port.ex** and **priv/z3.bl**.

Z3 4.16.0 VIA PORT DRIVER, PER QUERY

p50 = 2 ms · p99 = 3 ms (research/p13_smt)

The SMT backend is **blessed z3 as a native call**, not a reimplemented baby-SMT.

Because — the measured cost of the native call is two milliseconds; the measured cost of a baby solver is a correctness proof of a piece of code nobody needs. A home-grown SMT is a v2 option ONLY if z3-as-native fails — and 2 ms says it does not.

The payoff is rule proofs (§7’s foundation): given a rewrite rule like **(= x 0) → (zero? x)**, z3 either proves equivalence (**unsat** on the negation) or returns a countermodel. The countermodel is the valuable half: for this rule it is **tag_x = :string**, which is the derived assumption “this rule needs x : int” — the machine tells you what the rule secretly assumes (MVP-C, demo 03: two rules proven, one assumption derived, one broken rule caught).

miniKanren: relations and holes

miniKanren (a relational/logic-programming kernel, self-hosted in **priv/minikanren.bl**) runs relations **backwards**: given a type and a hole, synthesize what fits. Demo 04 types a term relation, leaves a hole in **((+ 1) _)**, and gets inhabitants: a numeral, a variable from the local context, a branch template. A string-shaped misfit gets none.

The hard-won mechanic: recursive goals without a delay diverge the search. The first kernel made delay the caller’s obligation — a footgun. The graduated module wraps every goal in delay **inside the conde / fresh / run macros**, so the footgun is not discouraged but **inexpressible**. (A **defsmell** could not express it; the macro could. That contrast — lint versus macro as the place to kill a footgun — is now a documented decision pattern.)

datom: the codebase as a database

priv/codebase.bl indexes source into datom facts — **:fn/name/ns/arity/line/ret-tag**, **:call/caller/callee/line** — and the query engine answers what grep cannot:

- **arity mismatches** and **unknown callees**, as queries;
- **reachability** and **impact** (reverse reachability: “what breaks if **t-meet** changes?” — 34 dependents when pointed at the checker itself);
- **find-by-type** (invariant L10): “every function returning **#{:string}**” — completion by type, the editor feature.

Indexing **priv/typed.bl** itself is the demo’s second act: the type checker’s own code, as data, answering questions about itself (MVP-B, demo 02).

Deodorant × types: promotion

Deodorant is beam-lisp’s lint tier: a registry of `{pattern ⇒ replacement}` smells, tiered **safe** (apply blind) or **idiomatic** (suggest). Types close the loop (MVP-E, demo 05):

TYPE-DISCHARGED PROMOTION

An idiomatic smell may declare its assumptions as data: `:needs '{?x int}` — the same assumption `z3` derives from the countermodel. Wherever the type environment **proves** the need (a guard narrowed `x` to `int`; the binding is a literal), the match is promoted to safe. Where it cannot, the match stays idiomatic. Unproven is never disproven: promotion only moves toward safety.

The demo’s four cases are the whole contract in miniature: promoted under `(int? x)`, promoted at a literal, held at an unannotated parameter, held at a disproven string.

Two mechanics from this layer are worth their own lines, because both are now registered smells themselves. First, `defsmell` grew `:needs` through its existing options **passthrough** — the registry absorbed the type system without a macro change. Second, building it exposed a real language-level trap: `index-of` returns `nil` when absent, and the BEAM’s term order makes `(>= nil 0)` **true**. That `nil-pun` is now a registered deodorant smell (`index-of-ge-zero`), found by this phase and fixed at the root in the same commit.

The Lean tier

Six capabilities borrowed from theorem provers, each demoed (P15):

1. **Evidence and hover** — the `[line col] → type` table, with elaborated answers. (Live since §5.2.)
2. **Holes and synthesis** — §6.2.
3. **The effect lattice** — `pure < atom < process < io`, unknown on top. A three-function chain ending in `swap!` infers `:atom` at every level, and a `^:pure` claim on the chain’s head is **rejected**, naming the effect and the line. Unknown callees are `:unknown` — you cannot claim purity over code you cannot see.
4. **Termination** — `(recur (dec i))` accepted, `(recur (inc i))` rejected with the elaborated form, `^{:decreasing (count xs)}` trusted **and reported** (the escape hatch is auditable, not silent). The discovered division of labour: the termination checker owns **shape** (structurally shrinking), the type checker owns **type** (a `dec` on a string crashes — that is §5’s warning, not a divergence).
5. **Deferred constraints** — the contract’s rule 2, working: a call checked before its callee exists waits; when a later namespace defines `^{:args [int]} helper`, the string argument is now provably wrong and warns; a never-defined callee is listed at DAG end as silent-unknown.
6. **^:opaque** — the reducibility knob. An opaque def is trusted at its declared signature, its body never walked; a transparent def whose body contradicts its declared `:ret` warns (“declared (int) but body returns (string)”). Trust is auditable: the signature table records `{:opaque true}`.

Proven live queries

datom's `watch` re-runs a query on every relevant commit and ships the `{:added :removed}` diff. MVP-F (`priv/live.bl`, demo 06) gates that on **monotonicity**:

`register-live!` refuses non-monotone queries, naming the offending clause. The callback is **not** purity-gated — delivery via `send` is a `:process` effect by nature — but its declared effect is reported at registration and any `^:pure` claim is provable file-wide. Because — with `[:not ...]`, a new datom can make an old answer false; a live query over negation silently becomes “the world changed under a non-monotone question”, and incremental consumers that cache or append break. Monotone queries only grow (modulo retractions, which the diff reports honestly).

The demo runs the whole story: a monotone query watches live (`:pending` stays silent, `:shipped` delivers `{:added #[[e]]}`), a negated query is refused with its clause, and a callback that **claims** `^:pure` while calling `erlang/send` is caught by the effect checker at line precision.

The capability map

Six graduated capabilities, each a module in `priv/`, each with a running demo:

MVP	capability	demo
A	the checker: lattice, positions, evidence, hover (<code>priv/typed.bl</code>)	01_check_demo
B	the codebase as a database (<code>priv/codebase.bl</code>)	02_codebase_demo
C	solver-backed rule proofs (<code>priv/z3.bl</code> + <code>z3_port.ex</code>)	03_solver_demo
D	holes and synthesis (<code>priv/minikanren.bl</code>)	04_holes_demo
E	deodorant × types: promotion (<code>priv/promote.bl</code>)	05_promote_demo
F	proven live queries (<code>priv/live.bl</code> + <code>effects.bl</code>)	06_live_demo

plus `priv/errors.bl` (§10), `priv/deferred.bl`, `priv/termination.bl`. The full regression is 663 language tests, 0 failures, with every demo in this document passing in the same process.

Errors: the five archetypes, one rendering

Every warning — from any layer — renders identically (P11, `priv/errors.bl`): file:line:col, the user's own line, a caret under the offending form, delaborated types. Widths use the token boundary because ends are not tracked; that approximation is documented in the code, not hidden.

```
$ mix beam_lisp.run examples/typing/07_errors_demo.bl
```

```
m.bl:2:19: double-it: argument 1 is (:string), declared (:int)
2 | (defn buggy [] (double-it "s"))
```



```

|           ^
file-a.bl:2:19: helper: argument 1 was deferred, now provably wrong: (:string) vs declared (:int)
  2 | (defn use-it [] (helper "s"))
    |           ^^^^^
t.bl:2:37: recur not provably shrinking: (recur (inc i))
  2 | (defn badcount [n] (loop [i n] (if (> i 0) (recur (inc i)) i)))
    |           ^

```

The L12 criterion is a predicate, not a hope: `errors/delaborated?` scans rendered output for machine-representation markers (`{:list`, `#{`, raw tuples) and fails the demo if any survives. The user never sees the machine’s notation — that is checked, not promised.

Optics: the verdict

P12 asked whether specter-style optics could be **the** rule language for the checkers, replacing the hand-rolled tree walkers. The measured answer is a split:

OPTICS OWN

Shape selection. One-line
paths replace forty-line walkers:
[(codewalk) (head "recur")] finds
every recur with proven parity.
Even scope cuts are expressible —
within #{ "loop" "fn" } visits a bound-
ary form but never enters it — because a cut
is a descent decision, and descent is what a
navigator is.

WALKERS OWN

Flow. Guard narrowing, let-environments, clause order — anything depending on context accumulated along the path. A predicate navigator sees a node, never an environment. Rewriting stays rewrite’s: the code navs are selection-only by design.

The navs live in `research/pl2_optics/` with a graduation plan item: first consumer is termination's `find-recurs`, when it next changes.

Tradeoffs: what this design gives up

Honesty section. Each of these is a real cost, chosen deliberately.

- **No Hindley–Milner inference.** Types are tags, not principal type schemes; there are no type variables, no unification over function types. Polymorphic higher-order code (`map`, `filter` over anything) types its results from seeds, not from the argument’s element type. A `map` of `inc` over ints yields `seq(any)`, not `seq(int)`. The lattice cannot express “the element type flows through” — miniKanren relations can, and bridging them is v2.
- **Silence is the price of soundness.** Unannotated code with unseeded calls checks clean even when wrong. P6 measured the residue on the spell seam: 63 of 113 evidence entries still `any` after seeding. The mitigation is the deferred-constraint machinery and curated seeds — both shrink the unknown set without ever guessing.
- **Seeds are maintained by hand.** 63 signatures, runtime-verified, sound-conservative. They rot if core changes; the check is a test, not a proof. Deriving them from AOT metadata is a v2 item.

- **Open-world sealing is load-order-dependent.** Multimethod and protocol checks are valid at a fingerprint; a method added after the DAG seals revokes warnings but no check re-runs. The AOT cache must carry fingerprints (§5.4) — designed, not yet wired.
- **Shapes originate at literals and schemas.** A map built by computation carries no shape. The `:keys` destructuring of a param-bound map is silent. `^{:shape ...}` annotations are the planned surface; the schema bridge (§5.6) covers queries.
- **The effect lattice has four rungs.** `pure < atom < process < io` with unknown on top. No effect polymorphism, no regions, no per-atom tracking. Enough to reject a lying `^:pure`; not enough to prove a scheduler fair.
- **z3 is a process.** Two milliseconds per query, a port, a binary to deploy. The baby-SMT alternative was measured and rejected (§6.1) — the cost is operational, not latency.

v2 directions

In priority order, each a natural extension of a shipped piece:

1. **Fingerprints in the AOT cache.** The open-world mechanism (§5.4) becomes the cache-invalidation key; a cached analysis replays only under a matching fingerprint. This unifies with FEAT-002’s content hashing.
2. **Element-type flow.** Seed polymorphism via one type variable per higher-order seed (`map : (a → b) → seq(a) → seq(b)`), discharged by the lattice where concrete, by miniKanren where not. This closes the largest measured `any` residue.
3. **`^{:shape ...}` annotations.** Param-bound maps enter with shapes; P8’s rules apply unchanged.
4. **Optics graduation.** `priv/code_navs.bl` lands when termination’s `find-recurs` refactors onto it; then promote and effects follow.
5. **Effect-aware scheduling.** The effect lattice is already precise enough to warn “this `receive` blocks a `^:pure` path”; the scheduler is the client.
6. **The backwards-synthesis loose end.** miniKanren’s backwards search is slow (goal ordering, likely); the timing harness exists at `research/p13_minikanren`. Not a blocker — sideways and forwards carry the demos — but it is the honest loose end of the logic tier.
7. **A `.olean`-style analysis artifact.** Lean ships compiled **analysis** beside compiled code; the evidence table, fingerprints, and proofs are exactly what such an artifact holds. The AOT cache is where it lands.

References

Elixir’s set-theoretic types. G. Castagna, V. Duboc, J. V. de Azevedo: the design documents and the incremental releases since Elixir 1.17. The philosophy §3 inherits is stated best in the Elixir documentation’s own “gradual set-theoretic types” reference.

The checker philosophy. Dialyzer’s “never wrong” discipline (success typings; Lindahl & Sagonas) is the twenty-five-year existence proof that sound-warnings-only works on the BEAM.

Positions through elaboration. Lean 4’s InfoTree — elaborated nodes stamped with the syntax they came from — is the direct model for §5.2’s threading desugaring and for the evidence table itself.

The Lean tier. §8’s six capabilities are Lean’s elaborator features (types-on-hover, holes, opaque definitions, decreasing measures, deferred constraints) re-cut for a dynamic Lisp: evidence instead of proof terms, warnings instead of kernels.

miniKanren. Friedman, Byrd, Kiselyov, **The Reasoned Schemer**; the kernel is the classic μ Kanren skeleton with beam-lisp’s delay made structural (§6.2).

SMT backend choice. z3 (de Moura & Bjørner), 4.16.0, driven as a native call at $p50 = 2$ ms per query (research/p13_smt). The alternative — a purpose-built baby solver — was evaluated and rejected on measured grounds: the native call’s cost is operational (a deployed binary, a port), while a home solver’s cost is **correctness risk in the proof layer itself**, the one place this design cannot afford it. If z3-as-native ever fails operationally, the port boundary is where a replacement slots in — the call sites know SMT-LIB, not z3.

Optics. Specter (Marz) for the navigator algebra; the van Laarhoven formulation (Pickering, Gibbons, Wu) for `priv/optics.bl`. P12’s split verdict — optics own shape, walkers own flow — is this phase’s own measurement.

Datalog. datom’s query engine (this repo, `priv/datom/query/`) — the codebase-as-database MVP stands on it, and monotonicity (§9) is datalog’s foundational theorem applied to live queries.